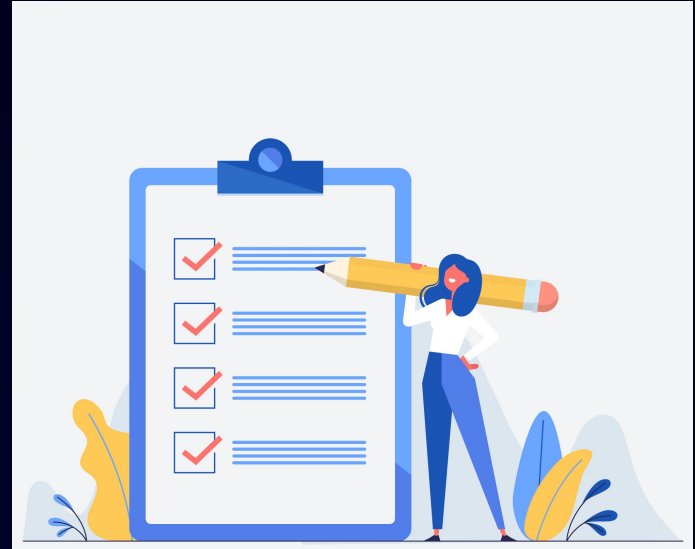




GitHub Collaboration, PR Workflow & Portfolio Deployment

Learning Objectives

- Understand and use Git workflow
- Create and manage Pull Requests
- Perform basic code reviews
- Resolve merge conflicts confidently
- Track tasks using Issues
- Contribute to open-source projects
- Deploy websites using GitHub Pages
- Build clean and professional repositories
- Explain projects and workflow clearly
- Practice and improve development skills





Pull Requests

What is a Pull Request (PR)?

A Pull Request (PR) is a way to propose changes to a codebase.

Commonly used in platforms like GitHub.

It allows developers to:

- Share code changes
- Request reviews
- Discuss improvements before merging
- PRs help maintain code quality and collaboration

Pull Request Workflow

Basic PR Flow:

1. Create a new branch
2. Make changes in the branch
3. Push branch to remote repository
4. Open a Pull Request
5. Team reviews code
6. Merge into main branch

Flow Visualization:

create branch → push → open PR → review → merge

Demo PR

Make a small change (e.g., edit README file)

Run:

git checkout -b feature-update

git add .

git commit -m "Updated README"

git push origin feature-update

Go to GitHub

Click “Compare & Pull Request”

Add:

- Title
- Description of changes
- Click Create Pull Request



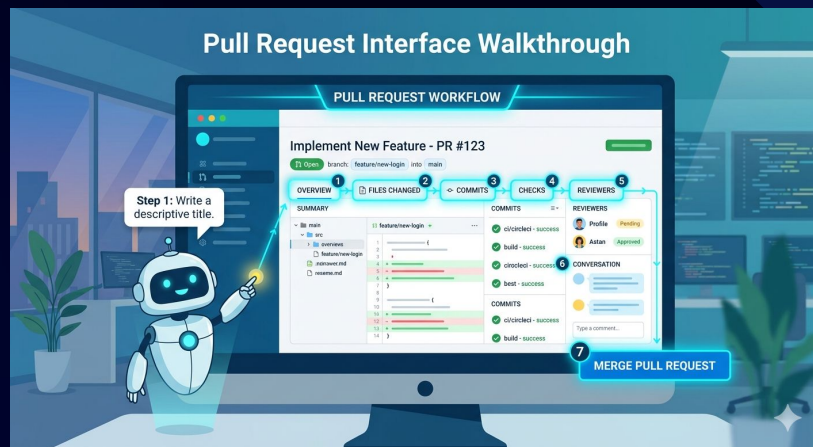
PR Interface Walkthrough

Key sections in a PR:

- Conversation Tab
- Discussion and comments
- Files Changed
- Shows code differences (diff view)
- Review Options
- Approve
- Request changes
- Comment
- Merge Button
- Merge after approval (e.g., “Merge Pull Request”)

Why it matters:

- Ensures code review
- Improves team collaboration
- Prevents bugs before merging



Pop Quiz

Q. What is the main purpose of a Pull Request (PR)?

A

**To directly delete code from
the main branch**

B

**To propose and review changes
before merging into the main
branch**

Pop Quiz

Q. What is the main purpose of a Pull Request (PR)?

A

**To directly delete code from
the main branch**

B

**To propose and review changes
before merging into the main
branch**



Code Review Basics

Why code review matters?

- Improves code quality before it reaches main branch
- Helps catch bugs and mistakes early
- Keeps codebase consistent across team
- Encourages knowledge sharing among developers

In short:

Better code + fewer issues + stronger team collaboration

How to Do a Code Review (On a PR)?

Using platforms like GitHub:

- Open the Pull Request
- Go to “Files Changed” tab
- Review line-by-line changes

Add comments:

- Click + next to a line
- Write feedback or questions

Suggest changes:

- Use “Suggest changes” option
- Provide improved code snippet

Submit review:

- Comment
- Approve
- Request changes



Simple Code Review Checklist

Keep it quick and practical:

- Readability
- Is the code easy to understand?
- Clear structure and formatting?
- Correctness
- Does it work as expected?
- Any edge cases missed?
- Naming
- Are variables and functions meaningful?
- Consistent naming style?

***Tip: Focus on important issues first,
avoid over-commenting on minor style
nitpicks***



Pop Quiz

Q. When reviewing a Pull Request, what should you focus on first?

A

**Only fixing indentation and
formatting**

B

**Checking if the code is correct,
readable, and well-named**

Pop Quiz

Q. When reviewing a Pull Request, what should you focus on first?

A

**Only fixing indentation and
formatting**

B

**Checking if the code is correct,
readable, and well-named**



Merge Conflicts on GitHub

What is a Merge Conflict?

- Happens when two branches change the **same part of a file**
- GitHub cannot automatically decide which change to keep
- Common in team collaboration when multiple people edit similar code

Key idea: Git needs your help to resolve the conflict



Conflict Scenario in a PR

- Developer A edits *line 10* in a file
- Developer B also edits *line 10* differently
- Both create Pull Requests
- First PR gets merged
- Second PR now shows a **merge conflict**

Result in PR:

- Warning: “This branch has conflicts that must be resolved”



Understanding Conflict Markers

When a conflict occurs, Git shows markers like:

<<<<<< HEAD

Your current branch changes

=====

Incoming branch changes

>>>>>> feature-branch

- <<<<<< HEAD → your version
- ===== → separator
- >>>>>> → incoming changes

Your task: Edit and keep the correct final version, then remove markers

How to Resolve Conflicts

Option 1: On GitHub (UI)

- Open PR → Click **“Resolve conflicts”**
- Edit code directly
- Mark as resolved → Commit

Option 2: Locally

```
git pull origin main  
# fix conflicts in code editor  
git add .  
git commit  
git push
```

Important Reminder:

- Conflicts are **normal in collaboration**
- They show active teamwork—not a mistake
- Always review carefully before merging



Pop Quiz

Q. What should you do when you see conflict markers in your code?

A

Review both versions, keep the correct code, and remove the markers

B

Delete all code between the markers

Pop Quiz

Q. What should you do when you see conflict markers in your code?

A

Review both versions, keep the correct code, and remove the markers

B

Delete all code between the markers



Issues – Task Tracking Basics

What are Issues?

- Issues are used as a **task and bug tracker** in GitHub
- Help teams organize work like:
 - Bug reports 🐛
 - Feature requests ✨
 - Tasks or improvements

Why use Issues?

- Keeps work **organized and visible**
- Helps track **progress and responsibility**
- Easy to link with Pull Requests



How to Use Issues (Basic Workflow)?

- **Create an Issue**
 - Go to “Issues” tab → Click **New Issue**
 - Add title + clear description
- **Assign it**
 - Select a team member responsible
- **Add Labels**
 - Example: bug, enhancement, documentation
- **Add Milestone (brief)**
 - Group issues under a goal (e.g., “Version 1.0”)

Tip: Keep issues short, clear, and actionable



Pop Quiz

Q. What is the main purpose of an Issue in GitHub?

A

To store code changes

B

To track tasks, bugs, and feature requests

Pop Quiz

Q. What is the main purpose of an Issue in GitHub?

A

To store code changes

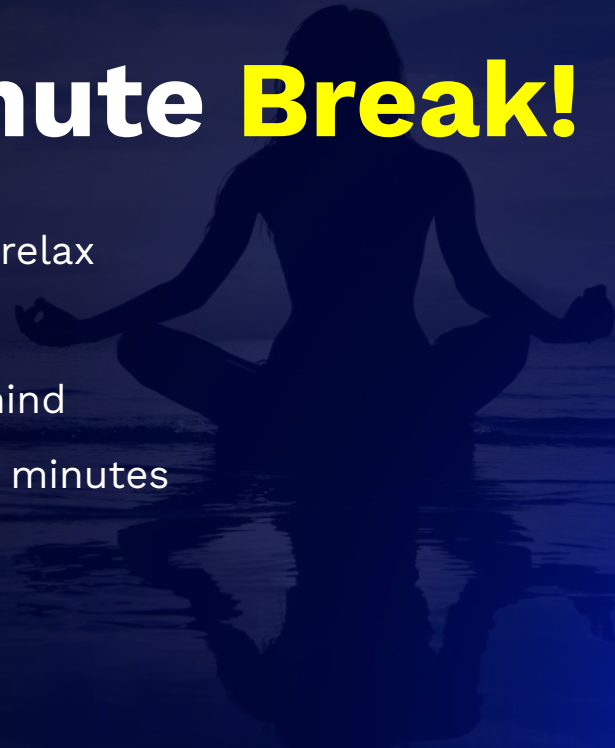
B

To track tasks, bugs, and feature requests



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes





Forking & Contribution Workflow

What is Forking?

- creating a **copy of someone else's repository** into your own account on GitHub
- You get full control to experiment without affecting the original project

Why fork?

- Contribute to open-source projects 
- Safely make changes without permission to the main repo
- Test ideas independently

Forking Workflow

1. Fork the repository
2. Clone your fork locally

```
git clone <your-fork-url>
```

3. Create a new branch

```
git checkout -b feature-change
```

4. Make changes + commit
5. Push to your fork
6. Open Pull Request to original repo

Flow Summary:

fork → clone → branch → changes → push → PR



Why This Matters (Open Source)?

- This is the **standard workflow for open-source contributions**
- Maintainers review your PR before merging
- Encourages:
 - Collaboration across the world 🌍
 - Code quality through review
 - Learning from real projects

Key Point: You don't need direct access to contribute!

Pop Quiz

Q. After pushing changes to your fork, what is the next step?

A

Delete the repository

B

**Open a Pull Request to the
original repository**

Pop Quiz

Q. After pushing changes to your fork, what is the next step?

A

Delete the repository

B

**Open a Pull Request to the
original repository**



GitHub Pages Deployment

What is GitHub Pages?

- GitHub Pages lets you host **static websites for free**
- Works directly from your repository on GitHub

Supports:

- HTML, CSS, JavaScript
- Portfolio sites, documentation, landing pages

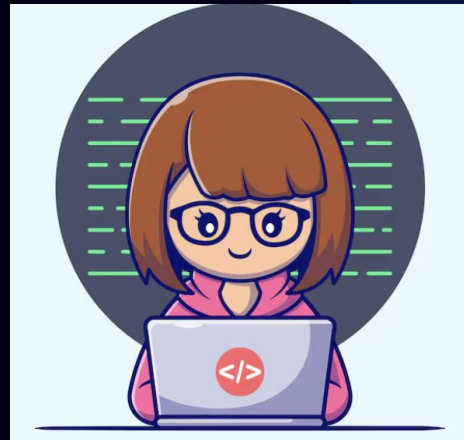
Key Benefit: No server setup needed 🚀

How GitHub Pages Works?

- You push website files to a repository
- GitHub builds and hosts the site automatically
- Generates a **live public URL**

Example URL format:

`https://username.github.io/repository-name/`



Steps to Deploy (Basic Setup)

1. Go to your repository
2. Click **Settings**
3. Find **Pages** section
4. Under “Source”:
 - Select **main branch**
 - Select **/root** folder
5. Click **Save**

That's it — deployment starts automatically



Access Your Live Website

- Wait a few seconds/minutes ⌚
- GitHub provides a live URL in Pages settings
- Open link to view your site

Tips:

- Make sure index.html exists
- Refresh if site doesn't load immediately

Reminder: Great for portfolios and demos!

Pop Quiz

Q. What type of websites can you host using GitHub Pages?

A

**Dynamic backend applications (e.g.,
with databases)**

B

**Static websites using HTML,
CSS, and JavaScript**

Pop Quiz

Q. What type of websites can you host using GitHub Pages?

A

**Dynamic backend applications (e.g.,
with databases)**

B

**Static websites using HTML,
CSS, and JavaScript**



Portfolio Repository Best Practices

Clean & Professional Repository Structure

On GitHub, your repo should look organized:

- Use a **clear folder structure**
 - /src, /assets, /docs (if needed)
- Keep files **well-organized and minimal**
- Use **proper file naming**
 - index.html, style.css, app.js
- Avoid clutter (unused files, random versions)

Goal: Make it easy to understand at a glance

Code Quality & Documentation

- Maintain a **meaningful commit history**
 - Example: “Add login page UI” (not “update stuff”)
- Add a strong **README.md**
 - Project description
 - Features
 - Tech stack
 - How to run
- Include **Live Link**
 - Deploy using GitHub Pages
 - Add URL in README

Tip: Treat your repo like a mini product

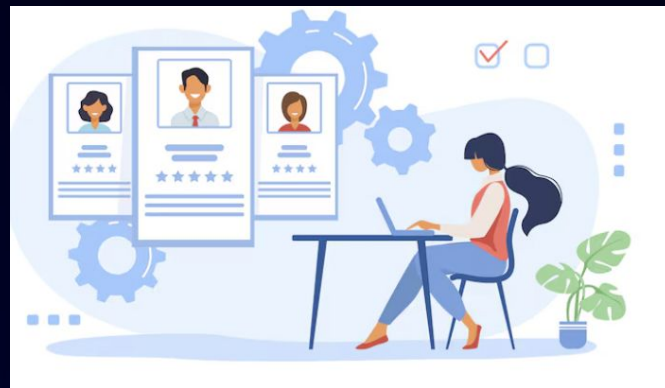
Why It Matters (For Recruiters)

- Recruiters often check your GitHub profile
- A clean repo shows:
 - Professionalism
 - Attention to detail
 - Real-world coding ability

Make sure:

- Project runs without errors
- Code is readable
- README explains everything clearly

Key Insight: Your GitHub = your portfolio 📁



Pop Quiz

Q. What makes a repository more attractive to recruiters?

A

Clean structure, clear commits, and a detailed README

B

Random commits and no README

Pop Quiz

Q. What makes a repository more attractive to recruiters?

A

Clean structure, clear commits, and a detailed README

B

Random commits and no README



Interview Prep Recap (High Value)

Core Skills Covered So Far

- **HTML Basics** → Structure of a webpage
- **CSS Fundamentals** → Styling and layout
- **CLI Basics** → Navigating and running commands
- **Git Workflow** → Branch → Commit → PR → Merge using GitHub

Continued

Be ready to explain

“Explain your project”

- What it does
- Technologies used

Key features

- “Explain the Git workflow you followed”
- How you created branches
- Made changes
- Opened and merged PRs



Future Strategy

Next Step:

- Practice regularly 
- Revise concepts 
- Build more small projects 

Goal: Be confident explaining and demonstrating your work!

Summary

In this lecture, we have covered:

- Pull Requests: Propose, review, merge changes
- Code Review: Improve quality, catch issues
- Merge Conflicts: Resolve overlapping code changes
- Issues: Track tasks and bugs
- Forking Workflow: Contribute via fork and PR
- GitHub Pages: Deploy static websites easily
- Portfolio: Clean, documented, recruiter-ready repos
- Final Skills: HTML, CSS, CLI, Git

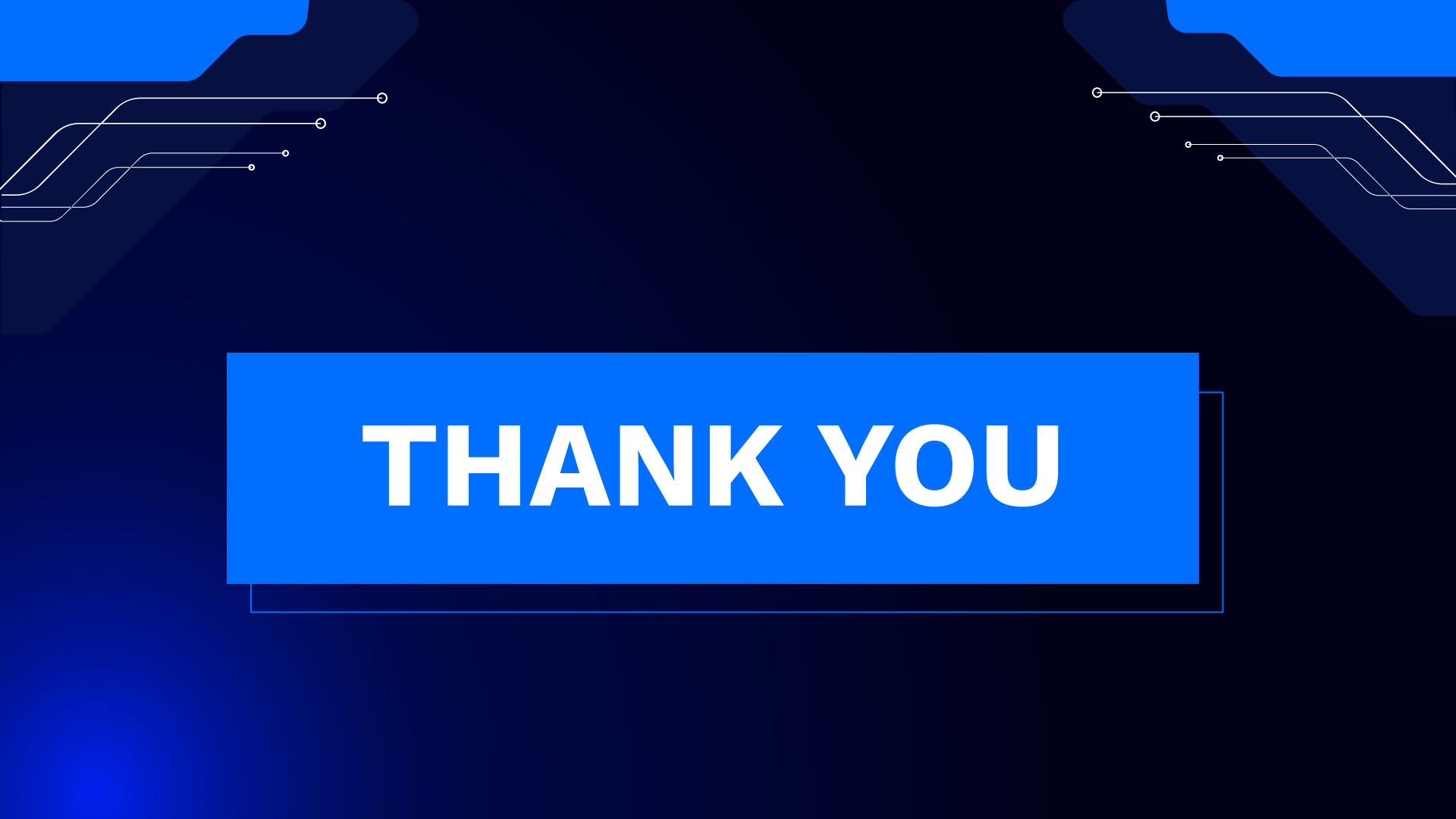




DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU